How Python stores values: names and objects

Objects in memory, names as arrows, what = shares, which types can change, and is versus ==.

THE QUESTION

After `b = a`, does changing `a` also change `b`?

THE INTUITION

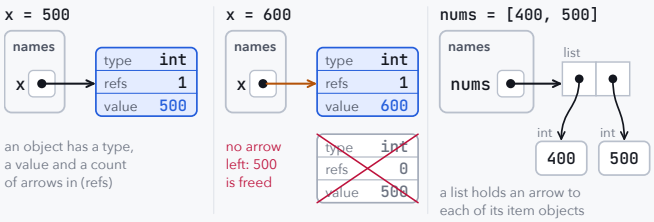
Every value is an object in memory. A name is only a label with an arrow to an object, and = moves that arrow without copying anything. Two names can point at one object.

KEY TAKEAWAY

`b = a` shares, it never copies. Rebinding moves one arrow; mutating changes the object that every arrow sees.

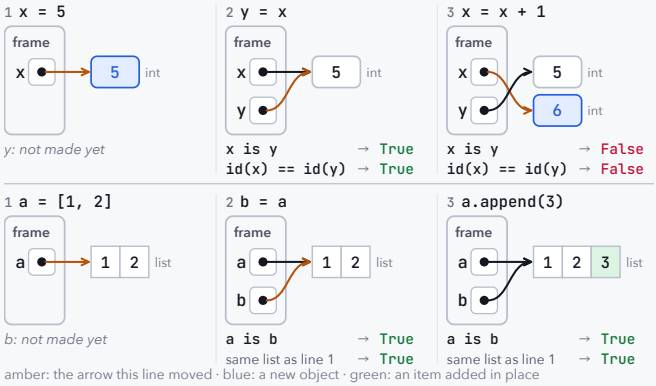
a = [1, 2] → an object in memory → a: an arrow to it → b = a: a second arrow → a.append(3) changes it → b sees [1, 2, 3]

1 Every value is an object; a name is an arrow



Python keeps each value as an **object** in memory, with a **type**, a **value** and a **reference count** (how many arrows point at it). A variable is a **name** in a **frame** with an arrow, a **reference**, to an object. `x = 600` makes a new object and moves the arrow. An object no arrow reaches is freed: **garbage collection**.

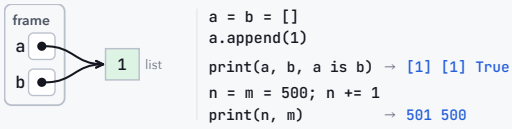
2 Rebinding an int versus changing a list



amber: the arrow this line moved · blue: a new object · green: an item added in place

`id(v)` is a number unique to one object while it exists (in standard Python, its memory address). Equal ids mean one object; `a is b` asks exactly that. Top: `x = x + 1` makes a new int and moves only `x` (**rebinding**): `print(x, y)` gives `6 5`. Bottom: `b = a` makes two **aliases** of one list; `a.append(3)` changes (**mutates**) it and moves no arrow, so `print(b)` gives `[1, 2, 3]`.

3 a = b = [] gives one list two names



`a = b = []` makes one list with both names, like the notebook's `a = b = c = 500` (one int, three names). For an int that is harmless: `n += 1` rebinds only `n`. For two lists, write two lines: `a = []` and `b = []`.

4 Only some types can change in place

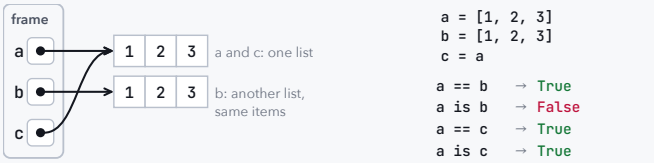
type	v	change item 0	v += ...	v is old	mutable
int	5	*no items*	v += 1	False	no
float	2.5	*no items*	v += 0.5	False	no
bool	True	*no items*	*gives an int*		no
str	'ab'	v[0] = 'X' TypeError	v += 'c'	False	no
tuple	(1, 2)	v[0] = 9 TypeError	v += (3,)	False	no
list	[1, 2]	v[0] = 9 [9, 2]	v += [3]	True	yes

`TypeError: 'str' object does not support item assignment`

v is old: is v still the same object after +=? False: += made a new one

A **mutable** object can be changed after it is made; an **immutable** one never can. So far only the list is mutable. A **tuple** is a list in round brackets that cannot change (the functions sheets use it). On an immutable value += builds a new object and rebinds the name, so another name never sees it. Dictionaries and sets, on later sheets, are mutable.

5 == compares values; is compares objects



line 1	line 2	x is y at >>>	x is y in a script
x = 256	y = 256	True	True
x = 257	y = 257	False	True
x = "hi there"	y = "hi there"	False	True

`a == b` asks whether two values are equal; `a is b` whether both names point at one object. Python may reuse one object for equal small ints and short strings (the lecture's cached ints, −5 to 256), but that is an implementation detail: the same lines answer differently at >>> and in a script. Use == for numbers and text; keep `is` for "same object?" and `x is None`.

6 b = a shares; a[:], list(a) and a.copy() copy

second list made by	b is a	b == a	a after b.append(99)
b = a	True	True	[1, 2, 3, 99]
b = a[:]	False	True	[1, 2, 3]
b = list(a)	False	True	[1, 2, 3]
b = a.copy()	False	True	[1, 2, 3]

each row starts again from a = [1, 2, 3]; red: a changed through b

A **copy** is a new list with the same items: `b == a` but not `b is a`, so `b.append(99)` leaves `a` alone (`a[:]` is from the slicing sheet). The notebook opens with the alias case: `y = x`, then `y[1] = 4` makes `print(x)` show `[1, 4, 3]`.

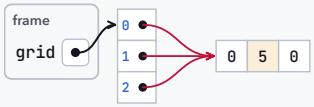
Copy on purpose, and deep enough

The list-of-lists trap, shallow and deep copies, += on a list, and a list passed to a function.

7 `[[0] * 3] * 3` repeats one row three times

```
grid = [[0] * 3] * 3
grid[0][1] = 5
```

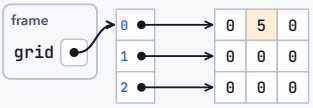
the trap: one row




```
[[0, 5, 0], [0, 5, 0], [0, 5, 0]]
grid[0] is grid[1] → True
```

```
grid = []
for r in range(3):
    grid.append([0] * 3)
grid[0][1] = 5
```

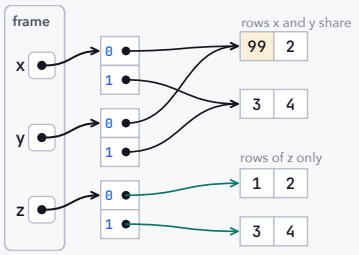
the fix: a new row each pass




```
[[0, 5, 0], [0, 0, 0], [0, 0, 0]]
grid[0] is grid[1] → False
```

`*` repeats a list's arrows, not the objects. For `[0] * 3` that is harmless: `row[1] = 5` replaces one arrow (`[0, 5, 0]`). `[[0] * 3] * 3` has three arrows to one row, as does the notebook's `[row, row, row]`. Make a new row on each pass of a loop, or write `[[0] * 3 for r in range(3)]`.

8 `copy()` is shallow; `copy.deepcopy()` copies the rows too




```
x = [[1, 2], [3, 4]]
y = x.copy()
z = copy.deepcopy(x)
y[0][0] = 99

y is x          → False
y[0] is x[0]    → True
z[0] is x[0]    → False

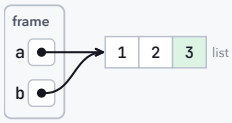
print(x) → [[99, 2], [3, 4]]
print(z) → [[1, 2], [3, 4]]
```

amber: the item set through y

A **shallow copy** (`x.copy()`, `x[:]`, `list(x)`) is a new outer list with arrows to the same inner lists: `y.append([5])` leaves `x` alone, but `y[0][0] = 99` changes a shared row. A **deep copy** copies every level: `import copy`, then `copy.deepcopy(x)`. A flat list of numbers needs only a shallow copy.

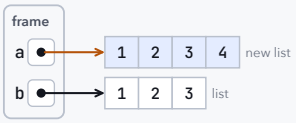
9 `a += [3]` changes the list; `a = a + [4]` makes a new one

```
a += [3]
```




```
a is b → True    b sees the 3
```

```
then a = a + [4]
```




```
a is b → False   b keeps [1, 2, 3]
```

run in this order after a = [1, 2]; b = a · green: added in place · blue: a new list

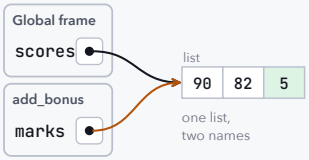
On a list, `a += [3]` works like `a.extend([3])`: it changes the object, so the alias `b` sees it. `a = a + [4]` builds a new list and moves only `a`: `print(a, b)` gives `[1, 2, 3, 4] [1, 2, 3]`. They differ exactly when another name shares the list. On an int, `n += 1` always rebinds.

10 A function gets the caller's list, not a copy

```
1 def add_bonus(marks):
2     marks.append(5)
3
4 scores = [90, 82]
5 add_bonus(scores)
6 print(scores)
```

```
print shows [90, 82, 5]
marks is scores → True
```

just before add_bonus returns:



A **function** made with `def` (the functions sheet) runs in its own frame; its **parameter** `marks` is a new name for the object passed in, so `marks.append(5)` changes `scores`. Rebinding it does not: with `marks = marks + [5]` inside, `print(scores)` gives `[90, 82]`. Pass `scores.copy()` to protect your list.

11 Common mistakes, after `a = [1, 2, 3]` and `x = [[1, 2], [3, 4]]`

you wrote	the last line Python prints; why, and the fix
`b = a; b.append(4); print(a)`	`prints [1, 2, 3, 4]` `b = a` is an alias. `b = a.copy()` keeps `a` at `[1, 2, 3]`
`n = 1000; print(n is 1000)`	`SyntaxWarning: "is" with a literal. Did you mean "=="?` `is` asks for one object: a script prints `True`, `>>>` gives `False`. Use `n == 1000`
`y = x.copy(); y[0][0] = 9; print(x)`	`prints [[9, 2], [3, 4]]` `copy()` shares the rows. `copy.deepcopy(x)` keeps `x` at `[[1, 2], [3, 4]]`
`y = copy.deepcopy(x)`	`NameError: name 'copy' is not defined` `copy` is a module: write `import copy` first

12 Try it (answers in green)

1. `p = [1, 2]; q = p; q.append(3)`, then `len(p)`?
   3: one list, two names.
2. `m = 7; n = m; m = m * 2`, then `n`?
   7: `m` moved, `n` did not.
3. `a = [1, 2]; b = [1, 2]`, then `a == b, a is b`?
   `True False`: equal, two lists.
4. `g = [[0] * 2] * 2; g[0][0] = 1`, then `g`?
   `[[1, 0], [1, 0]]`: one shared row.
5. `x = [[1], [2]]; y = x[:]; y[0].append(9)`, then `x`?
   `[[1, 9], [2]]`: the rows are shared.
6. `a = [1]; b = a; a += [2]; a = a + [3]`, then `a, b`?
   `[1, 2, 3] [1, 2]`

a name is an arrow · = never copies · rebinding moves an arrow, mutating changes the object · == values, `is` objects

Source: the memory allotment notebook and the handwritten lecture notes on memory, nipunbatra.github.io/comp22; draw your own diagrams at pythontutor.com. Every output comes from running the code with Python 3.11.11, and every diagram was drawn from `is` checks on the real objects. Next: the strings in depth sheet.